Competitive Programming Notes

Pakin Olanraktham

December 19, 2025

Contents

1 Data Structures **3**

1.1 Wavelet Tree 3

2 Dynamic Programming Optimization **4**

2.1 Convex Hull Trick 4

2.1.1 Inserting 4

2.1.2 Querying 5

2.2 Li-Chao Tree 6

2.2.1 Insertion 6

2.2.2 Query 7

2.3 Divide and Conquer 8

3 Tree Algorithms **9**

3.1 Centroid Decomposition 9

3.1.1 Centroid 9

3.1.2 Centroid Decomposition 9

4 Graph Algorithms **11**

4.1 DSU with Rollback 11

4.2 Maximum Flow 12

4.2.1 Edmonds-Karp 12

1 Data Structures

1.1 Wavelet Tree

Wavelet Tree is a Data Structure mainly for finding the k'th minimum value in some range $[l, r]$. The tree is built dynamically meaning that we will use pointers.

```
struct Wavelet {
    vector <int> p;
    Wavelet *l, *r;
};
```

When we build the tree, we check if the b'th bits is 0 or 1. If it's 0, we put that element into the left side. Otherwise, we put in the right side. The vector `p` is a prefix sum array counting number of elements in the left side.

```
void build(const vector <int> &A, int b) {
    if (b == 0 || A.empty()) return;
    l = new Wavelet();
    r = new Wavelet();
    int c = 0;
    vector <int> L, R;
    for (int x : A) {
        if (x & (1 << (b-1))) R.emplace_back(x);
        else L.emplace_back(x), c++;
        p.emplace_back(c);
    }
    l->build(L, b-1);
    r->build(R, b-1);
}
```

When we query, we count the number of elements in the left side in range l to r. If the number of elements in the left side is more than or equal to k, we recursive to the right side. Otherwise we go to the right side. We have to change the value of l, r, k too when moving down to the right side.

```
int qr(int l, int r, int k, int b) { // [l, r), k, 0-index
    if (b == 0 || p.empty()) return 0;
    int pr = (r > 0 ? p[r-1] : 0);
    int pl = (l > 0 ? p[l-1] : 0);
    int cnt = pr - pl;
    if (k < cnt) return this->l->qr(pl, pr, k, b-1);
    else return (1 << (b-1)) + this->r->qr(l-pl, r-pr, k-cnt, b-1);
}
```

Note that these functions are inside the `Wavelet` struct.

2 Dynamic Programming Optimization

2.1 Convex Hull Trick

Convex Hull Trick is an algorithm to find $\min_{j<i}(f_j(x))$ or $\max_{j<i}(f_j(x))$ efficiently.

First, we convert the function $f(x)$ to $mx + c$, which x depends only on i, m and c depends only on j. For example

```
dp[i] = max(dp[j] + x[i]*y[i] - x[j]*y[i] - a[i])
      = max((-x[j])*y[i] + dp[j]) + x[i]*y[i] - a[i]
```

As we can see, m = -x[j], x = y[i], c = dp[j]. Note that if we want to do CHT, the slope **must** be monotonic. Otherwise, we have to use Li-Chao Tree.

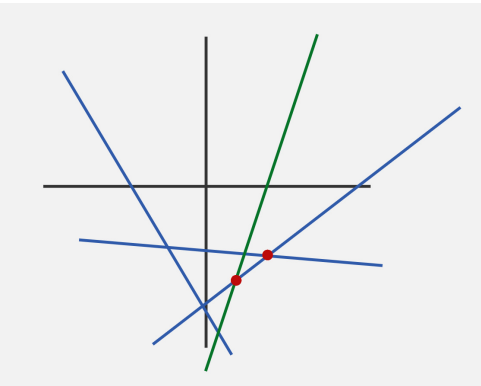
Now, how do we compute the maximum or minimum? We know that the recurrence is a line with monotonic slope.

2.1.1 Inserting

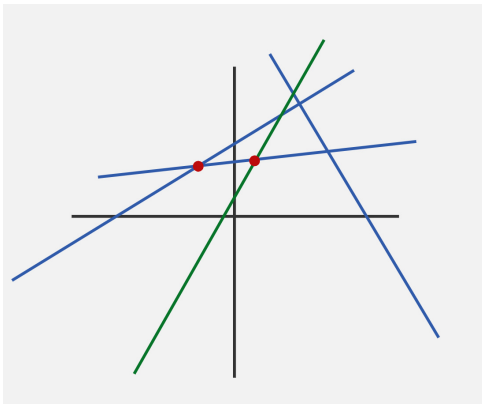
We will use `deque <int>` to store the lines indices.

For computing maximum, we want to maintain the lines as lower hull shaped. For minimum, we want the lines as upper hull shaped.

Before inserting, we have to check whether the old line is useless or not. We can do that by checking the intersection point between the [new line, top] and [top, top-1]. (You should try drawing on paper by yourself.)



Lower Hull (Maximum)



Upper Hull (Minimum)

For finding the intersection point we can do

```
double m(int x, int y) {
    return (double) (c[x] - c[y]) / (m[y] - m[x]);
}
```

2.1.2 Querying

Monotone Query

If the hull is maintained correctly for each type of query, we will always query from front for non-decreasing queries, and always from back for non-increasing queries.

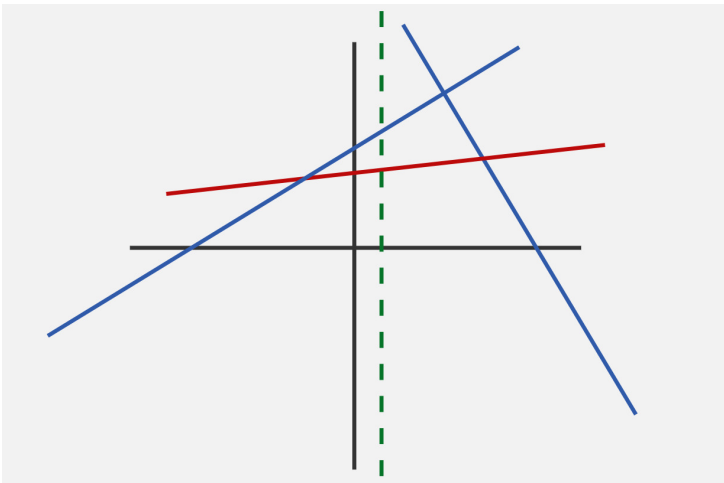
We will check the top and top-1. For maximum, if $f_{\text{top}}(x) \leq f_{\text{top-1}}(x)$, we pop the top. For minimum, if $f_{\text{top}}(x) \geq f_{\text{top-1}}(x)$, we pop the top. For example

```
while (sz > 1 && calc(cht[0], p[i]) <= calc(cht[1], p[i])) {
    cht.pop_front(), sz--;
}
```

Then the answer will be $f_{\text{top}}(x) + c$ where c is a constant that depends only on x.

Arbitrary Query

We perform a binary search for the line that contain x.



```
int l = 0, r = sz-2, idx = sz-1;
while (l <= r) {
    int mid = l + (r-l)/2;
    if (m(cht[mid], cht[mid+1]) <= x) l = mid+1;
    else r = mid-1, idx = mid;
}
```

2.2 Li-Chao Tree

Reference : A Simple Introduction to Li-Chao Segment Tree

Li-Chao Tree is an upgrade of Convex Hull Trick. Instead of monotone slope in CHT, it can handle both arbitrary slope and query.

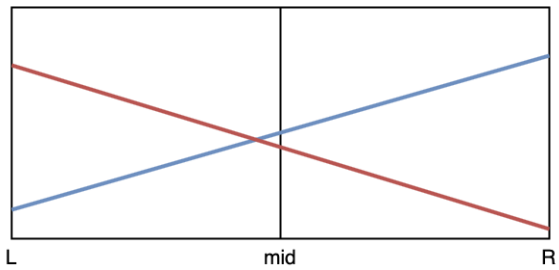
Given 2 types of queries, add a line to multiset S, find the maximum value at some point x. It can handle both of the query with $\mathcal{O}(\log X)$ where X is the maximum value of type-2 query. We will use a Segment Tree to store the best line for some range $[l, r]$. We can even do lazy propagation, sparse (dynamic node creation), or persistent with it.

We will use each node of the segment tree as a Line.

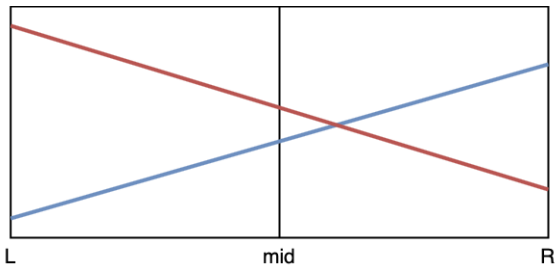
```
struct Line {
    ll m = 0, c = LLONG_MIN; // c = LLONG_MAX for min Li-Chao Tree
    ll operator()(ll x) { return m * x + c; }
} seg[4*mxX];
```

2.2.1 Insertion

For conveinence, we assume $m_{\text{new}} < m_{\text{old}}$ (if not, then swap the lines). We will consider 2 cases.



Case 1



Case 2

Case 1 – $f_{\text{new}}(\text{mid}) < f_{\text{old}}(\text{mid})$: The old line is better for range $[l, r]$, except some range in the left side, so we recursively insert the new line to the left.

Case 2 – $f_{\text{new}}(\text{mid}) > f_{\text{old}}(\text{mid})$: The new line is better for range $[l, r]$, except some range in the right side, so we recursively insert the old line to the right.

```
void ins(int l, int r, Line x, int u) {
    if (l == r) {
        if (seg[u](l) < x(l)) seg[u] = x;
        return;
    }
    int m = l + (r-l)/2;
    if (x.m > seg[u].m) swap(x, seg[u]);
    if (x(m) < seg[u](m)) ins(l, m, x, u<<1);
    else swap(x, seg[u]), ins(m+1, r, x, u<<1|1);
}
```

2.2.2 Query

We just query for point x normally, but we have to get the maximum with the current line too.

```
ll qr(int l, int r, int x, int u) {
    if (l == r) return seg[u](x);
    else {
        int m = l + (r-l)/2;
        if (m >= x) return max(seg[u](x), qr(l, m, x, u<<1));
        else return max(seg[u](x), qr(m+1, r, x, u<<1|1));
    }
}
```

2.3 Divide and Conquer

Divide and Conquer is also a technique we can use to optimize dynamic programming.

Consider this DP formula

$$\text{dp}[k][i] = \min_{j \le i}(\text{dp}[k-1][j-1] + C(j,i))$$

such that we can calculate $C(i,j)$ in $\mathcal{O}(1)$. If we solve this normally, it would take $\mathcal{O}(KN^2)$.

We can use Divide and Conquer technique to optimize this DP formula. Let $\text{opt}(k,i)$ be the optimal value of j that minimize the value of the RHS of the equation. This techinque will **only work** if $\text{opt}(k,i-1) \le \text{opt}(k,i)$ for all i, which is hard to prove.

From Quadrangle Inequality, if we found out that $C(a,c)+C(b,d) \le C(a,d)+C(b,c)$ satisfies for all $a < b < c < d$, then $\text{opt}(k,i-1) \le \text{opt}(k,i)$ holds (the proof of this has been left as an exercise to the reader).

Since calculating $\text{dp}[k][i]$ doesn't need other value in $\text{dp}[k]$, so we can calculate each i independently. This is the code for the algorithm.

```
void dnc(int l, int r, int optl, int optr) { // [l, r] is the
    current range, [optl, optr] is the range we need to check.
    if (l > r) return;
    int m = l + (r-l)/2, opt;
    dp[k][m] = INT_MAX;
    for (int i = optl; i <= optr; i++) {
        if (dp[k][m] > dp[k-1][i-1] + C(i, m)) {
            dp[k][m] = dp[k-1][i-1] + C(i, m);
            opt = i;
        }
    }
    dnc(l, m-1, optl, opt); // for all i < m, opt(k, i) <= opt(k, m)
    dnc(m+1, r, opt, optr); // for all i > m, opt(k, i) >= opt(k, m)
}
```

3 Tree Algorithms

3.1 Centroid Decomposition

3.1.1 Centroid

A centroid of a tree is defined as a node such that if removed, no connected component has more than $\frac{N}{2}$ nodes.

We can find a centroid in a tree by selecting an arbitrary node in the tree. Then loop through its children's subtree. If the size of a child's subtree is more than $\frac{N}{2}$, traverse down that node. Otherwise, if no child's subtree has size more than $\frac{N}{2}$, then that node is the centroid.

```
void fsz(int u, int p) {
    sz[u] = 1;
    for (auto v : adj[u]) if (v != p) fsz(v, u), sz[u] += sz[v];
}

int ct(int u, int p) {
    for (auto v : adj[u]) if (sz[v] * 2 > n) return ct(v, u);
    return u;
}
```

Properties

1. If the centroid of a tree isn't unique, there are exactly two centroids which are adjacent. Removing the edge between them divides the tree into two connected components of the equal sizes.
2. When a leaf is added to or removed from a tree, the centroid changes by at most one node.
3. When two trees are connected, the centroid of the resulting tree lies on the path between the centroids of the original trees.

3.1.2 Centroid Decomposition

Centroid Decomposition is a divide-and-conquer technique for trees, which recursively splits the tree by removing its centroid. The resulting decomposition tree has a height of at most $\mathcal{O}(\log N)$.

Properties

1. The height of the decomposed tree is at most $\log N$
2. Every path between u and v can be decomposed into two parts: u to $\text{lca}_{ctd}(u, v)$ and $\text{lca}_{ctd}(u, v)$ to v. We can use this property for querying and updating. In the **Xania and Tree** problem, we update and query by traversing up to the $\log N$ parents for each node.

```
int fsz(int u, int p) {
    sz[u] = 1;
    for (auto v : adj[u]) if (v != p && !vis[v]) sz[u] += fsz(v, u);
    return sz[u];
}

int ct(int u, int p, int tsz) {
    for (auto v : adj[u]) if (v != p && !vis[v]) if (sz[v] * 2 > tsz
        ) return ct(v, u, tsz);
    return u;
}

void ctd(int u, int p) {
    int C = ct(u, u, fsz(u, u));
    vis[C] = 1;
    if (p == -1) p = C;
    par[C] = p;
    for (auto v : adj[C]) if (!vis[v]) ctd(v, C);
}
```

4 Graph Algorithms

4.1 DSU with Rollback

DSU with Rollback is like a normal DSU but after a change, we can undo it. We will maintain the value before changes in a vector, $(\text{idx}, \text{val}_{\text{before}})$. If we want to undo a change, we just set that value back from the back of the vector, then pop_back() the vector.

```
int par[mxN], sz[mxN];
vector <pair <int, int>> rpar, rsz;

int fi(int x) {
    if (x == par[x]) return x;
    else return fi(par[x]);
}

void mg(int u, int v) {
    u = fi(u), v = fi(v);
    if (sz[u] < sz[v]) swap(u, v);
    rsz.emplace_back(u, sz[u]);
    sz[u] += sz[v];
    rpar.emplace_back(v, par[v]);
    par[v] = u;
}

void rollback() {
    sz[rsz.back().first] = rsz.back().second; rsz.pop_back();
    par[rpar.back().first] = rpar.back().second; rpar.pop_back();
}
```

Note that we don't use path compression because it would be slower and harder, updating multiple parents at a time.

4.2 Maximum Flow

4.2.1 Edmonds-Karp

Edmonds-Karp is an algorithm to find maximum flow in $\mathcal{O}(VE^2)$. We will find the shortest path with BFS from source to sink, then update the capacity of the edge, and do this until we can't reach the sink by using only the edges with capacity more than zero.

```
cin >> n >> m;
while (m--) {
    int u, v; ll w; cin >> u >> v >> w;
    adj[u].emplace_back(v);
    adj[v].emplace_back(u); // to undo a flow with reversed edge
    cap[u][v] += w;
}
auto bfs = [&] () {
    queue <int> q;
    memset(vis, 0, sizeof(vis)), memset(par, 0, sizeof(par));
    q.emplace(1), vis[1] = 1;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        if (u == n) return 1;
        for (int v : adj[u]) if (cap[u][v] > 0 && !vis[v]) vis[v] =
            1, q.emplace(v), par[v] = u;
    }
    return 0;
};
ll ans = 0;
while (bfs()) {
    vector <int> v;
    v.emplace_back(n);
    ll mn = LLONG_MAX;
    while (v.back() != 1) {
        // get the bottleneck of the path
        mn = min(mn, cap[par[v.back()]][v.back()]);
        v.emplace_back(par[v.back()]);
    }
    for (int i = 0; i < v.size()-1; i++) {
        // update the capacity of the edge
        cap[v[i+1]][v[i]] -= mn;
        cap[v[i]][v[i+1]] += mn; // to undo a flow
    }
    ans += mn;
}
cout << ans << '\n';
```